УНИВЕРСИТЕТ ИТМО

Факультет Систем Управления и Робототехники

ЛАБОРАТОРНАЯ №1
ПОСТРОЕНИЕ МАТЕМАТИЧЕСКОЙ МОДЕЛИ ДВИГАТЕЛЯ EV3

Аннотация – В данной работе проведено экспериментальное исследование динамики двигателя Lego EV3. Построена математическая модель двигателя в виде апериодического звена первого порядка. Определены статический коэффициент передачи и постоянная времени. Проведено сравнение экспериментальных данных с результатами моделирования в Simulink.

Ключевые слова: ДПТ; Идентификация; Python; MATLAB; Simulink

Выполнили

Ануфриев А. С.
Ндава Маргарет Чити
Глаголев М.К.
465029, @me_and_no_oneelse
505413, @margaritaanybody
504722, @R07331821

Проверил

Овчаров Алексей Олегович

25.03.2026

Содержание

1 Постановка задачи **2**
1.1 Цель и задачи исследования 2
1.2 Объект исследования 2
1.3 Математическая модель двигателя 2

2 Экспериментальная установка и методика **4**
2.1 Схема экспериментальной установки 4
2.2 Описание оборудования 5
2.3 Подключение и управление 5
2.4 Методика сбора данных 5

3 Идентификация параметров модели **6**
3.1 Методика определения параметров 6
3.2 Результаты 6

4 Анализ результатов **9**
4.1 Сводные результаты 9
4.2 Сравнение результатов 10

5 Моделирование в Simulink **15**
5.1 Описание структурной схемы 15
5.2 Результаты моделирования 15

6 Выводы **18**

7 Программы **19**
7.1 Программа EV3 19
7.2 MATLAB Program 19

1 Постановка задачи

1.1 Цель и задачи исследования

Исследуется динамика двигателя EV3. Задачи:

1. Снять переходные процессы при $\pm 20\%, \pm 40\%, \pm 60\%, \pm 80\%, \pm 100\%$.
2. Определить k и T_m модели.
3. Построить модель в Simulink.
4. Сравнить эксперимент, аппроксимацию и Simulink.
5. Выявить нелинейности.

1.2 Объект исследования

Объектом исследования является электродвигатель постоянного тока (ДПТ) с постоянными магнитами, используемый в образовательном конструкторе Lego Mindstorms EV3 (Large Motor). Физическая схема взаимодействия основных элементов двигателя показана на рис. 1. Ротор (якорь) состоит из обмоток и сердечника. Статор представляет собой неподвижный постоянный магнит. При подаче напряжения U на обмотки ротора возникает ток I, который, взаимодействуя с полем статора, создает электромагнитный момент M_{el}, приводящий ротор во вращение с угловой скоростью ω. В процессе вращения в обмотках наводится противо-ЭДС ε_i, которая препятствует изменению тока.

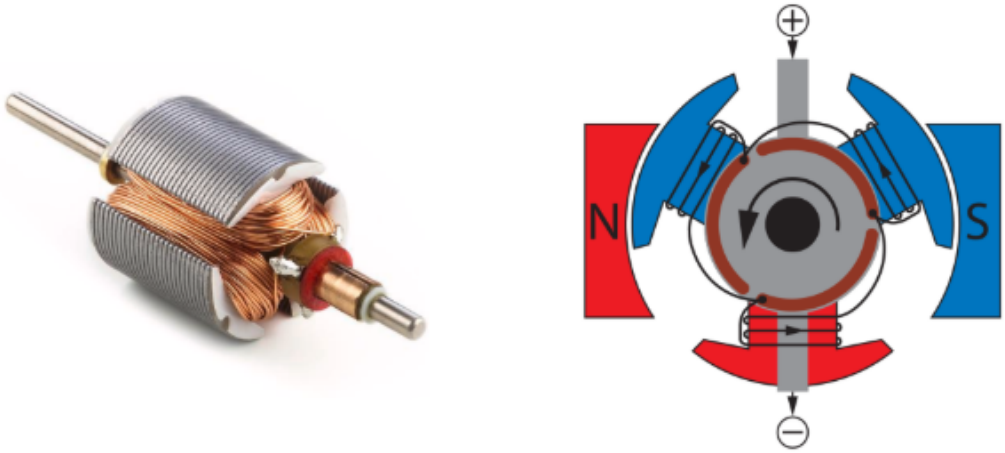


Рис. 1: Физическая схема двигателя постоянного тока.

1.3 Математическая модель двигателя

Для описания динамики двигателя постоянного тока с постоянными магнитами рассмотрим два основных физических закона.

Второй закон Ньютона для вращательного движения: Суммарный вращающий момент, действующий на ротор, равен произведению момента инерции на угловое ускорение:

$$J\frac{d\omega(t)}{dt} = M_{\text{em}}(t) - M_{\text{нагр}}(t), \tag{1}$$

где J — момент инерции ротора (кг·м^2), ω — угловая скорость (рад/с), M_{em} — электромагнитный момент двигателя (Н·м). В данной работе двигатель не нагружен, поэтому $M_{\text{нагр}} = 0$.

Электромагнитный момент пропорционален току якоря:

$$M_{\text{em}}(t) = k_m I(t), \tag{2}$$

где k_m — коэффициент момента (Н·м/А), I — ток в обмотках (А).

Закон Ома для цепи якоря: При вращении ротора в обмотках наводится противо-ЭДС $\varepsilon_i = -k_e\omega(t)$, препятствующая изменению тока. С учётом этого закон Ома принимает вид:

$$U(t) - k_e\omega(t) = I(t)R, \tag{3}$$

где U — поданное на двигатель напряжение (В), R — сопротивление обмоток (Ом), k_e — коэффициент противо-ЭДС (В·с/рад).

Вывод дифференциального уравнения: Выразим ток из (3):

$$I(t) = \frac{U(t) - k_e\omega(t)}{R}. \tag{4}$$

Подставим в (2), а затем в (1):

$$J\frac{d\omega(t)}{dt} = k_m \cdot \frac{U(t) - k_e\omega(t)}{R}. \tag{5}$$

Раскроем скобки и перенесём слагаемое с ω в левую часть:

$$J\frac{d\omega(t)}{dt} + \frac{k_m k_e}{R}\omega(t) = \frac{k_m}{R}U(t). \tag{6}$$

Разделим обе части на J:

$$\frac{d\omega(t)}{dt} + \frac{k_m k_e}{JR}\omega(t) = \frac{k_m}{JR}U(t). \tag{7}$$

Введение постоянной времени и коэффициента передачи: Введём электромеханическую постоянную времени T_m и статический коэффициент передачи по напряжению k_U:

$$T_m = \frac{JR}{k_m k_e}, \qquad k_U = \frac{1}{k_e}. \tag{8}$$

Тогда уравнение (7) принимает классический вид апериодического звена первого порядка:

$$T_m\frac{d\omega(t)}{dt} + \omega(t) = k_U \cdot U(t). \tag{9}$$

Переход к управляющему сигналу в процентах: В экспериментальной установке напряжение подаётся в процентах от максимального ($U_{\max} = 9$ В). Обозначим $u(t)$ — управляющий сигнал (%). Тогда:

$$U(t) = \frac{u(t)}{100}U_{\max}. \tag{10}$$

Подставляя в (9), получаем:

$$T_m\frac{d\omega(t)}{dt} + \omega(t) = k \cdot u(t), \tag{11}$$

где

$$k = \frac{U_{\max}}{100 \cdot k_e} \quad (\text{рад}/(\text{с}\cdot\%)). \tag{12}$$

Решение для ступенчатого входного сигнала: При подаче постоянного напряжения $u(t) = U_0 \cdot \mathbf{1}(t)$ решение уравнения (11) имеет вид:

$$\omega(t) = kU_0 \left(1 - e^{-t/T_m}\right), \tag{13}$$

а угол поворота получается интегрированием скорости:

$$\theta(t) = \int_0^t \omega(\tau) d\tau = kU_0 \left(t - T_m \left(1 - e^{-t/T_m}\right)\right). \tag{14}$$

Таким образом, динамика двигателя EV3 в первом приближении описывается передаточной функцией апериодического звена:

$$W(s) = \frac{k}{T_m s + 1}. \tag{15}$$

2 Экспериментальная установка и методика

2.1 Схема экспериментальной установки

Экспериментальная установка (рис. 2) состоит из контроллера EV3, к порту B которого подключен исследуемый двигатель Large Motor. Управление осуществляется с персонального компьютера.

Рис. 2: Общий вид экспериментальной установки.

2.2 Описание оборудования

Использовались: контроллер EV3, двигатель Large Motor, ПК для управления.

2.3 Подключение и управление

Контроллер подключался по SSH, скрипт передавался через scp, управление — через библиотеку `ev3dev-lang-python`.

2.4 Методика сбора данных

Для каждого уровня напряжения ($U \in \{\pm 20\%, \pm 40\%, \pm 60\%, \pm 80\%, \pm 100\%\}$) двигателю подавалось напряжения на 1 сек. Одновременно записывались: время t (мс), угол поворота $\theta(t)$ (град), угловая скорость $\omega(t)$ (град/с). Результаты сохранялись в TXT-файлы.

3 Идентификация параметров модели

3.1 Методика определения параметров

Идентификация параметров модели k и T_m производилась по экспериментальным данным угла поворота $\theta(t)$ с использованием метода наименьших квадратов (МНК) в среде MATLAB. Для аппроксимации использовалась теоретическая зависимость
$\hat{\theta}(t) = kU_0\left(t - T_m(1 - e^{-t/T_m})\right)$.

МНК минимизирует сумму квадратов отклонений экспериментальных точек θ_i от модельной кривой $\hat{\theta}(t_i)$:

$$S(k, T_m) = \sum_{i=1}^{N} \left(\theta_i - \hat{\theta}(t_i, k, T_m)\right)^2 \to \min \tag{16}$$

После нахождения параметров k и T_m для каждого уровня напряжения, статический коэффициент передачи k также был проверен по установившемуся значению скорости ω_{ust}: $k = \omega_{ust}/U_0$. Качество аппроксимации оценивалось с помощью коэффициента детерминации R^2, который показывает, какая доля дисперсии экспериментальных данных объясняется моделью:

$$R^2 = 1 - \frac{\sum(\theta_i - \hat{\theta}_i)^2}{\sum(\theta_i - \bar{\theta})^2} \tag{17}$$

3.2 Результаты

На рис. 3-5 показаны зависимости параметров модели от управляющего сигнала. Полученные значения сведены в табл. 1.

Анализ полученных данных позволяет сделать вывод о нелинейности исследуемого объекта. Видно, что коэффициент передачи k и постоянная времени T_m не являются константами, а возрастают с увеличением подаваемого напряжения. Это особенно заметно для T_m, которая при 100% мощности (0.1046 с) почти в 1.5 раза выше, чем при 20% (0.0656 с). Такое поведение может объясняться влиянием насыщения магнитной системы, изменением сопротивления обмоток при нагреве или нелинейностью механических характеристик (например, сил трения), которые в упрощенной модели не учитываются.

Таблица 1: Параметры передаточной функции двигателя для различных уровней управляющего сигнала

ШИМ (%)	k **(рад/с/%)**	T_m **(с)**
−100	0.1835	0.0912
−80	0.1822	0.0935
−60	0.1785	0.0759
−40	0.1733	0.0708
−20	0.1578	0.0634
+20	0.1652	0.0656
+40	0.1829	0.0747
+60	0.1889	0.0784
+80	0.1928	0.0883
+100	0.1967	0.1046
Среднее	**0.1802**	**0.0806**

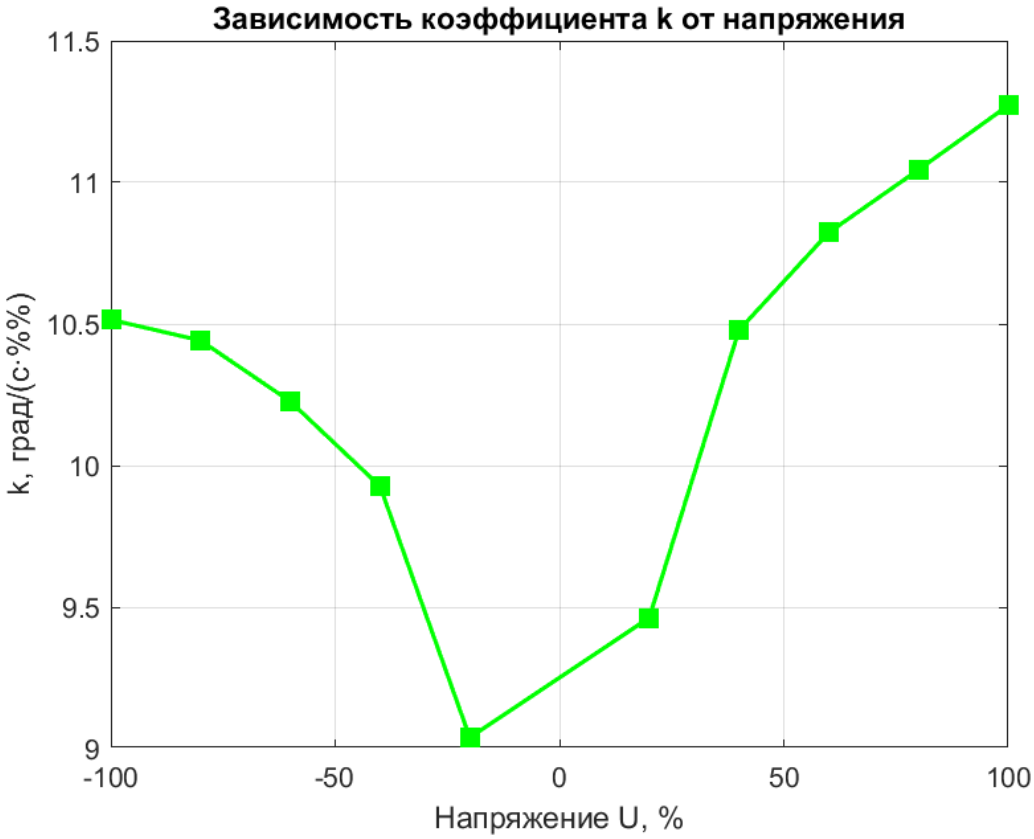


Рис. 3: Зависимость коэффициента k от напряжения

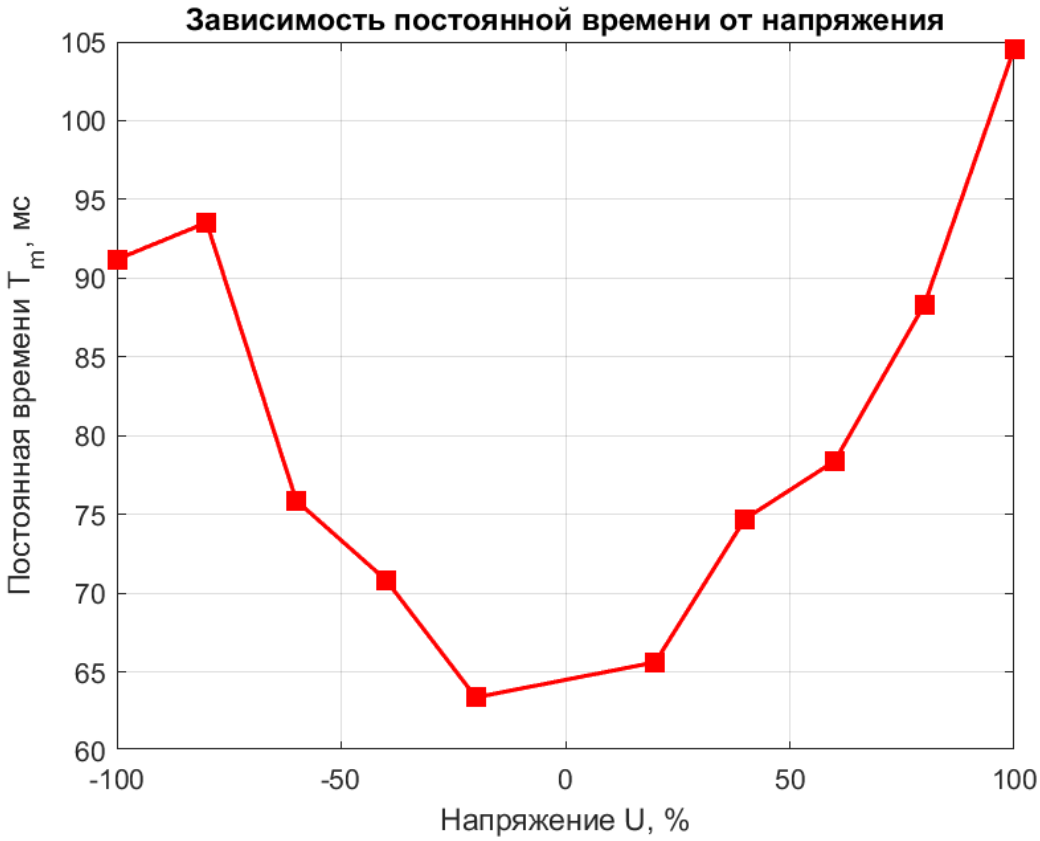


Рис. 4: Зависимость постоянной времени T_m от напряжения

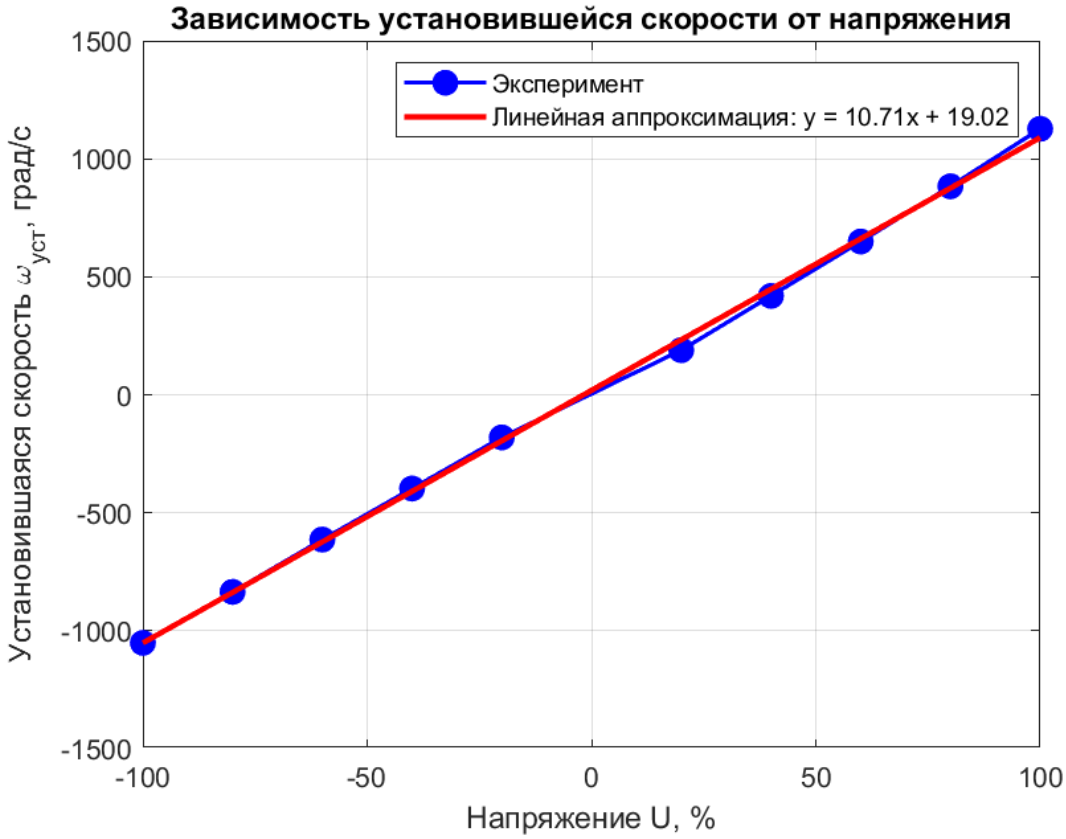


Рис. 5: Установившаяся угловая скорость в зависимости от напряжения

4 Анализ результатов

4.1 Сводные результаты

На рис. 6 и 7 представлены графики для всех пяти положительных и пяти отрицательных напряжений, позволяющие оценить общий характер переходных процессов.

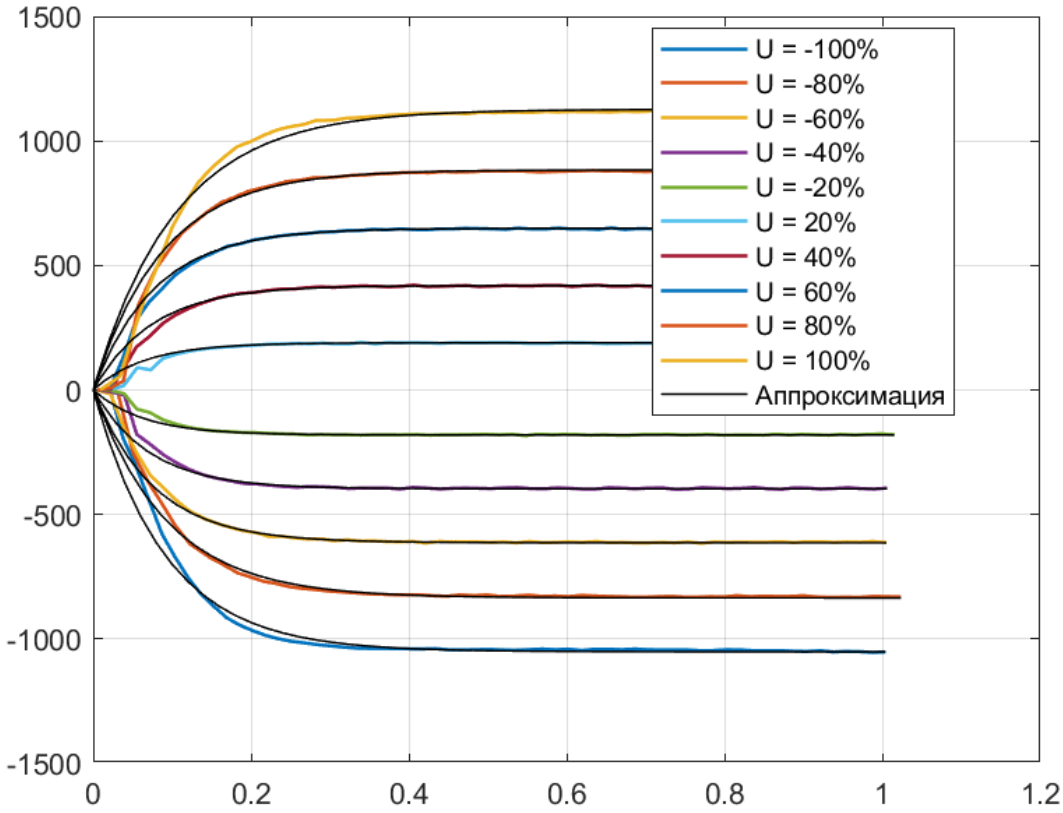


Рис. 6: Сравнение экспериментальных данных и моделирования в Simulink для угловой скорости при различных уровнях управляющего сигнала.

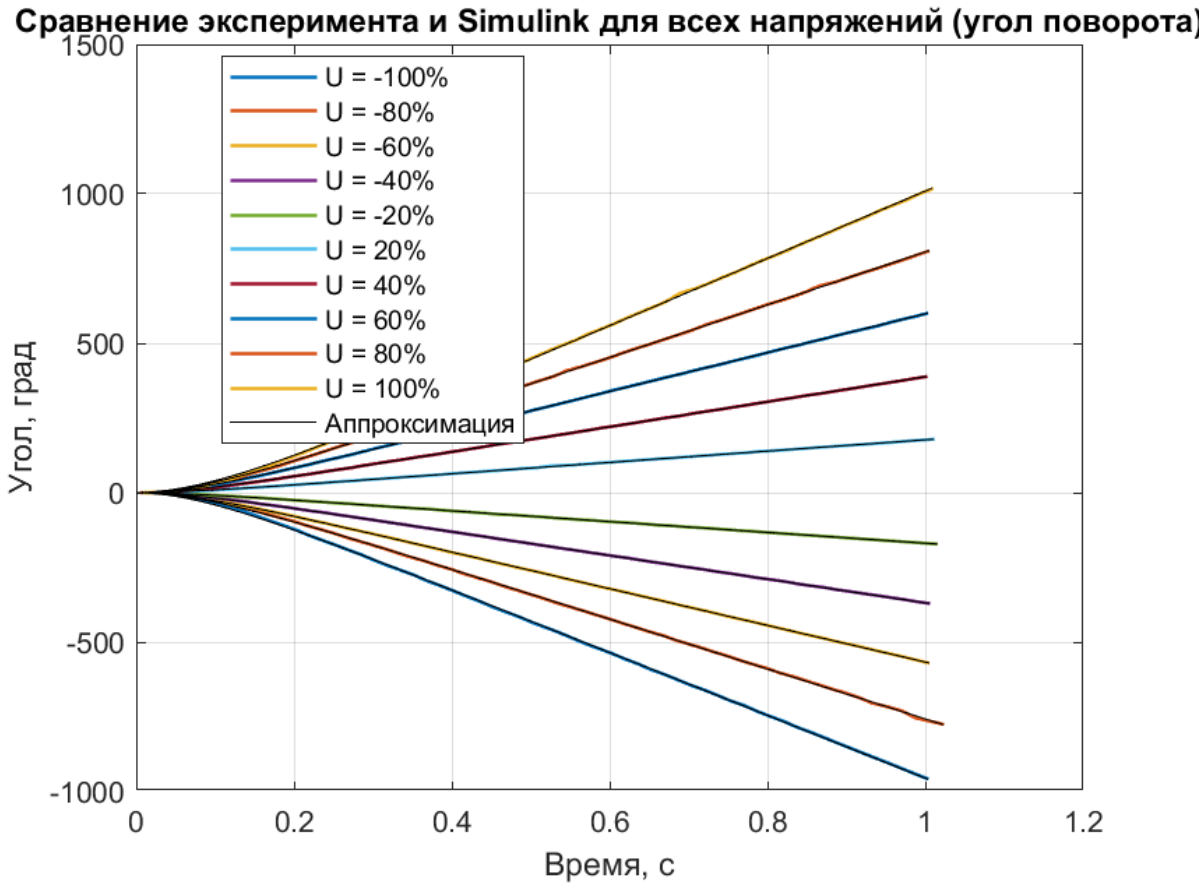
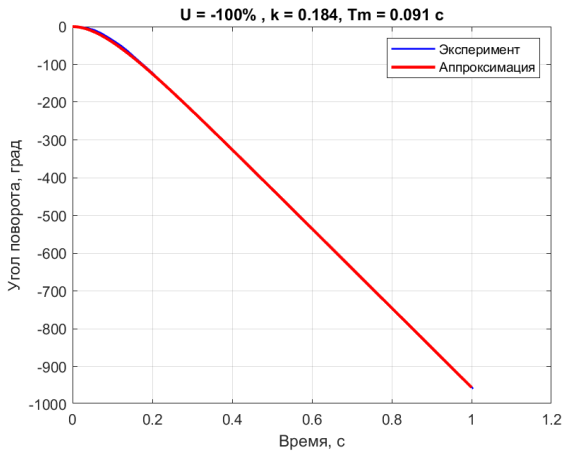


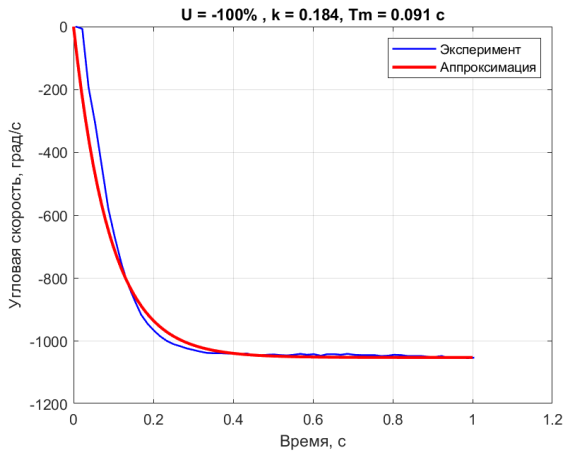
Рис. 7: Сравнение экспериментальных данных и моделирования в Simulink для угла поворота при различных уровнях управляющего сигнала.

4.2 Сравнение результатов

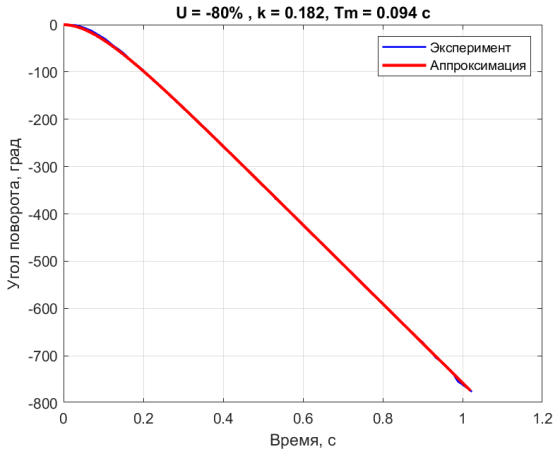
На рис. 8-11 показаны углы и скорости для всех режимов: синие точки — опыт, красная линия — апроксимация.



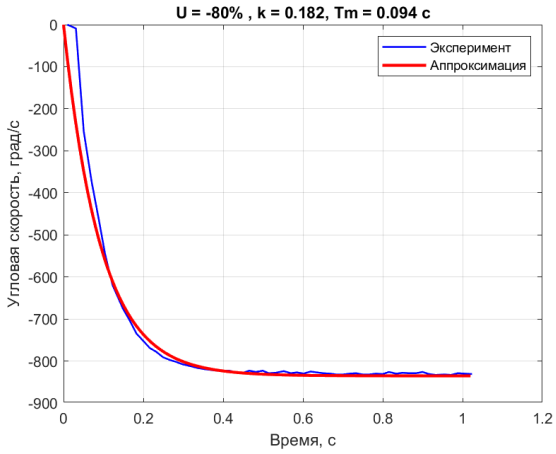
(a) Угол при -100%



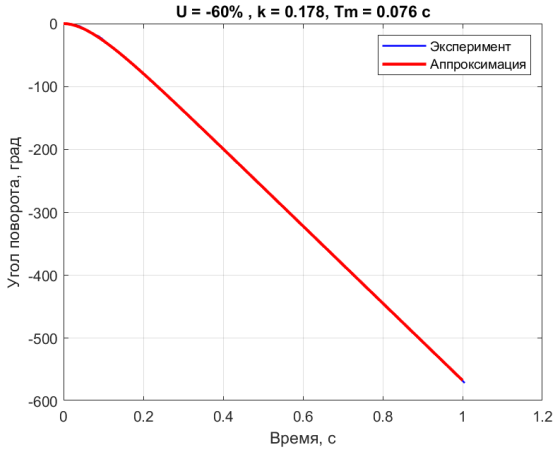
(b) Скорость при -100%



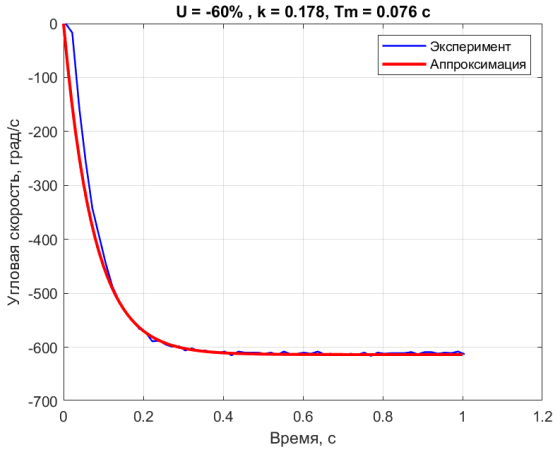
(c) Угол при -80%



(d) Скорость при -80%

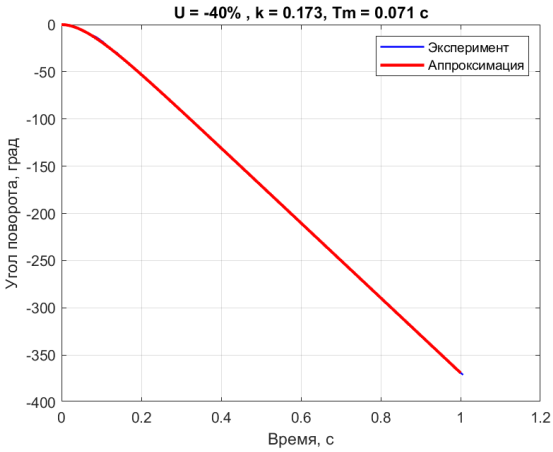


(e) Угол при -60%

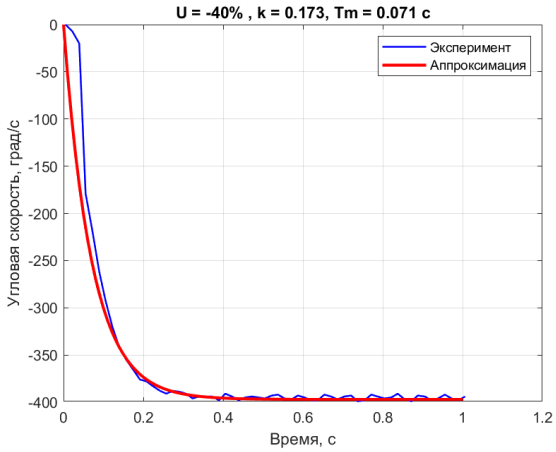


(f) Скорость при -60%

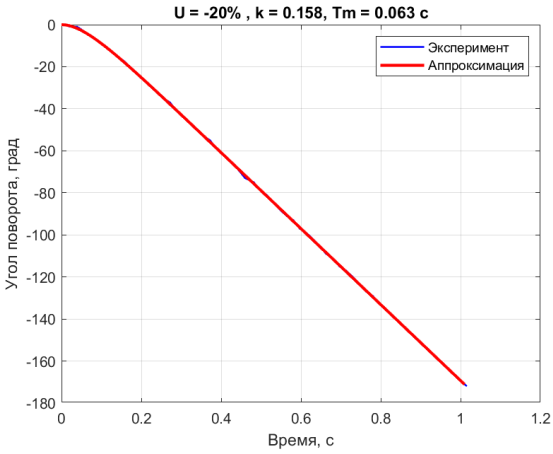
Рис. 8: Для максимальных напряжений график растет быстро, что делает кривизну переходного процесса менее заметной.



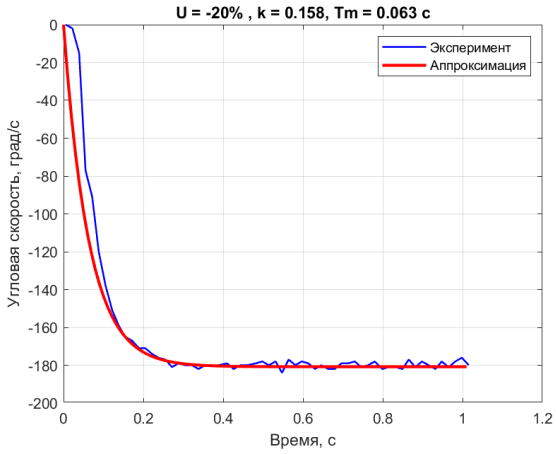
(a) Угол при -40%



(b) Скорость при -40%

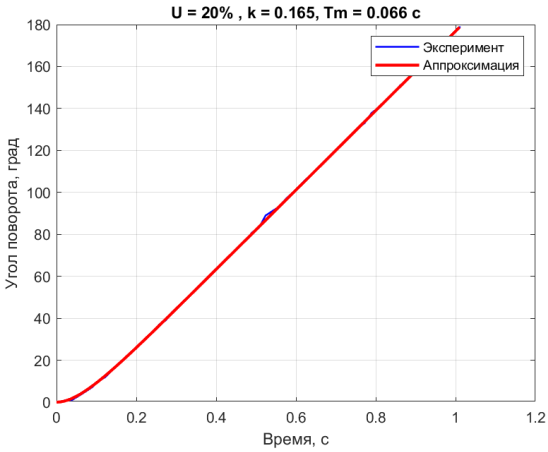


(c) Угол при -20%

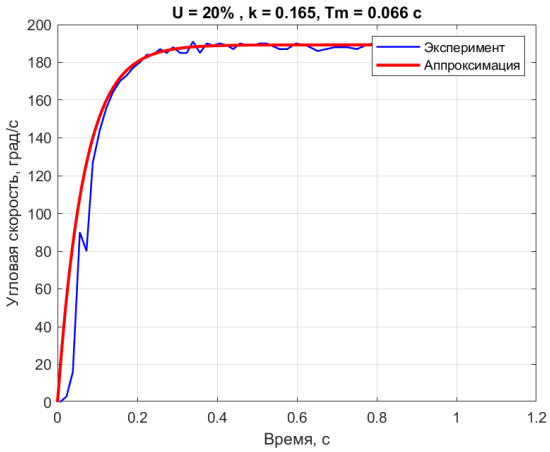


(d) Скорость при -20%

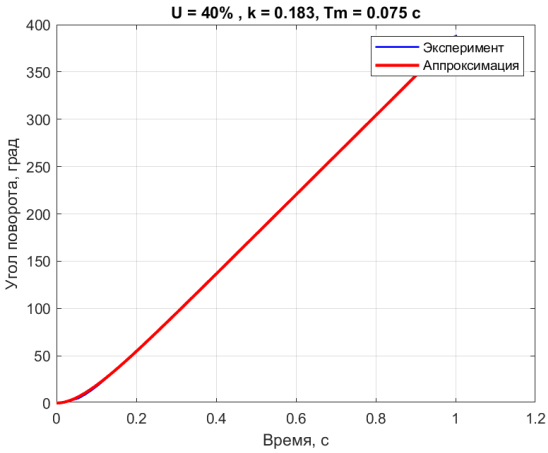
Рис. 9: Средние значения.Кривизну хорошо видно



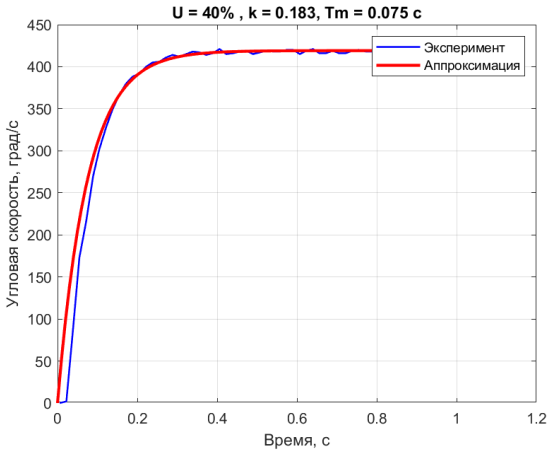
(a) Угол при +20%



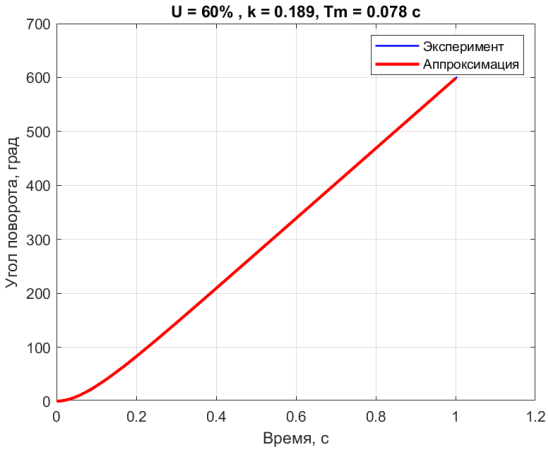
(b) Скорость при +20%



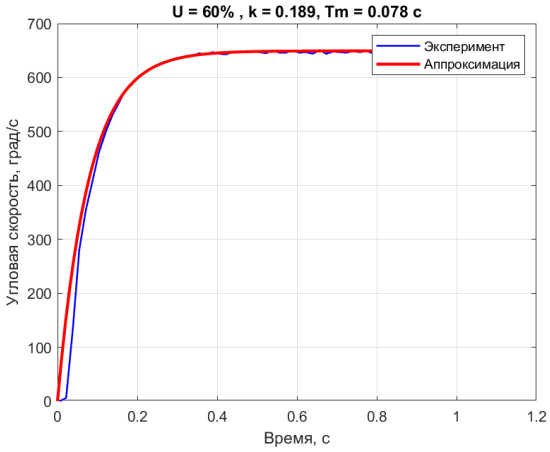
(c) Угол при +40%



(d) Скорость при +40%

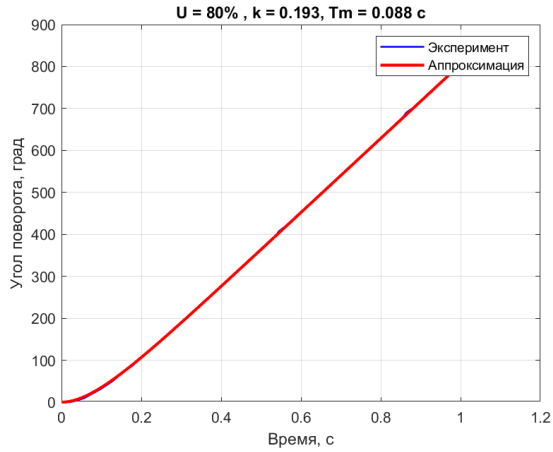


(e) Угол при +60%

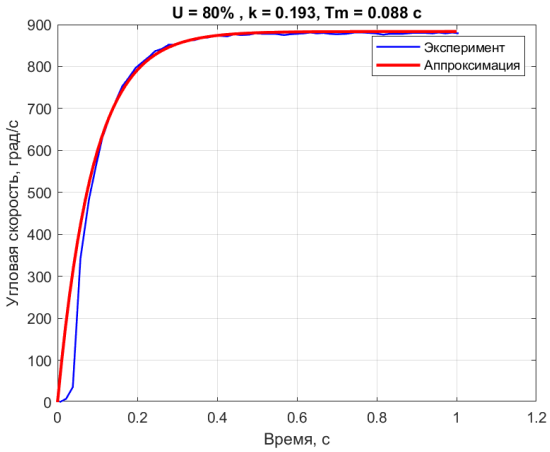


(f) Скорость при +60%

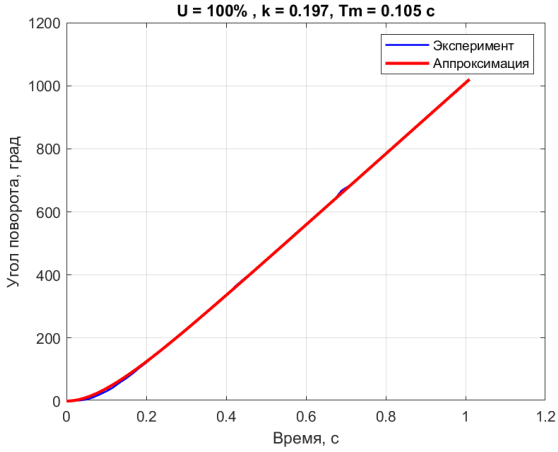
Рис. 10: Средние значения.



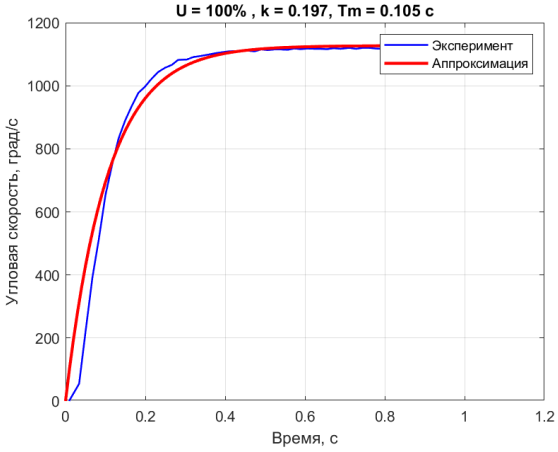
(a) Угол при +80%



(b) Скорость при +80%



(c) Угол при +100%



(d) Скорость при +100%

Рис. 11: Для максимальных напряжений график растет быстро, что делает кривизну переходного процесса менее заметной.

5 Моделирование в Simulink

Для верификации полученной математической модели было проведено имитационное моделирование в среде Simulink. Структурная схема модели (рис. 12) построена на основе дифференциального уравнения $T_m\dot{\omega}+\omega = ku(t)$ и позволяет получить зависимости угловой скорости $\omega(t)$ и угла поворота $\theta(t)$ от времени.

5.1 Описание структурной схемы

Схема (рис. 12) реализована следующим образом:

- Блок `Constant` задает значение управляющего сигнала U (%).
- Блок `Gain` с коэффициентом k формирует установившееся значение скорости $\omega_{уст} = kU$.
- Сумматор `Sum` вычитает из $\omega_{уст}$ текущее значение скорости $\omega(t)$, поступающее с выхода интегратора, формируя тем самым сигнал ошибки $\omega_{уст} - \omega(t)$.
- Блок `Gain1` с коэффициентом $1/T_m$ масштабирует сигнал ошибки в ускорение $\dot{\omega}$.
- Интегратор `Integrator1` интегрирует ускорение и возвращает текущее значение скорости $\omega(t)$, замыкая контур обратной связи.
- Второй интегратор `Integrator` интегрирует скорость и возвращает угол поворота $\theta(t)$.
- Блоки `To File` служат для сохранения результатов моделирования в файлы `omega_data.mat` и `theta_data.mat`.

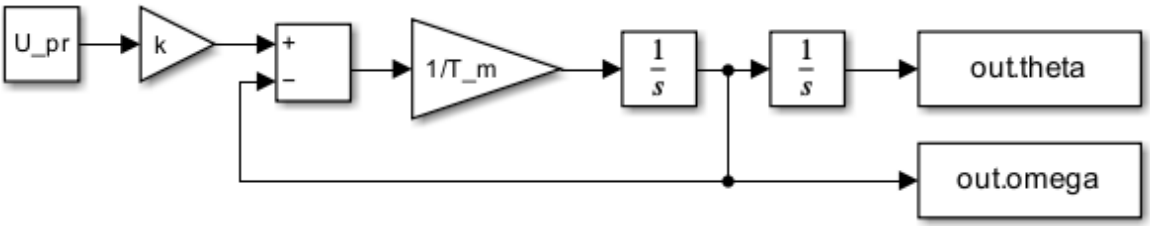
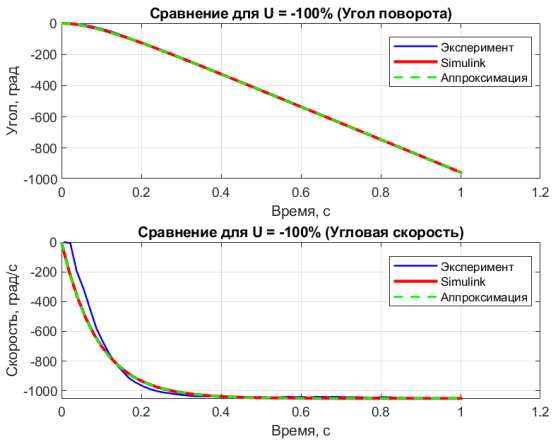


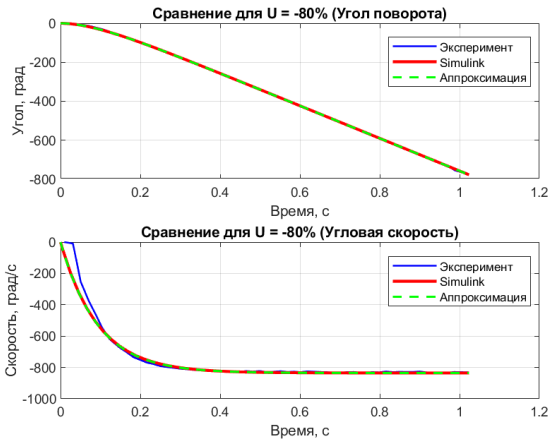
Рис. 12: Структурная схема модели двигателя EV3 в Simulink.

5.2 Результаты моделирования

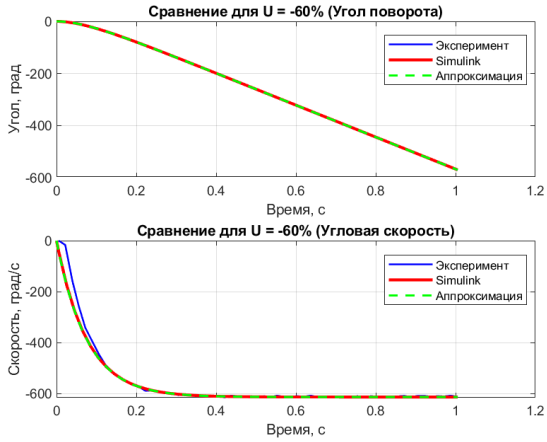
На рис. 13 и 14 представлено сравнение результатов моделирования в Simulink с экспериментальными данными и аппроксимирующей кривой. Синие точки обозначают экспериментальные данные, красная линия — аппроксимирующую кривую, зеленая линия — результаты моделирования в Simulink.



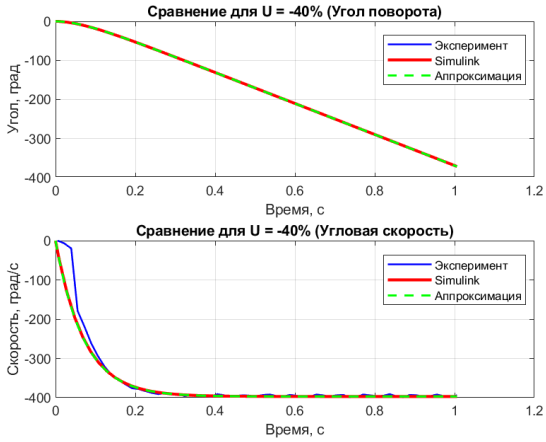
(a) Сравнение при -100%



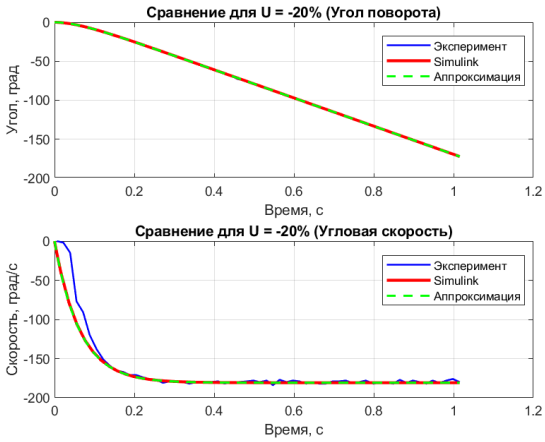
(b) Сравнение при -80%



(c) Сравнение при -60%

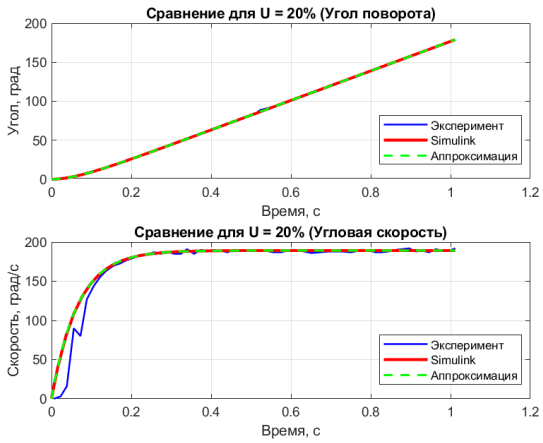


(d) Сравнение при -40%

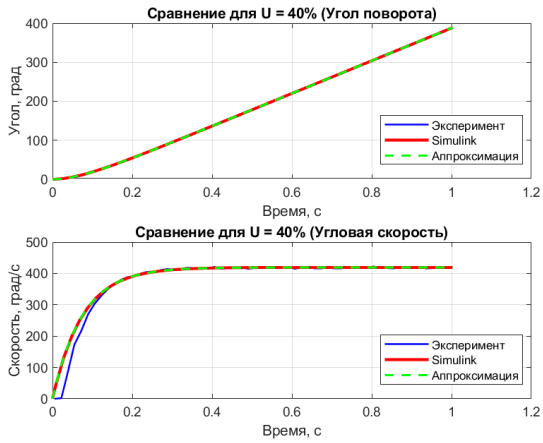


(e) Сравнение при -20%

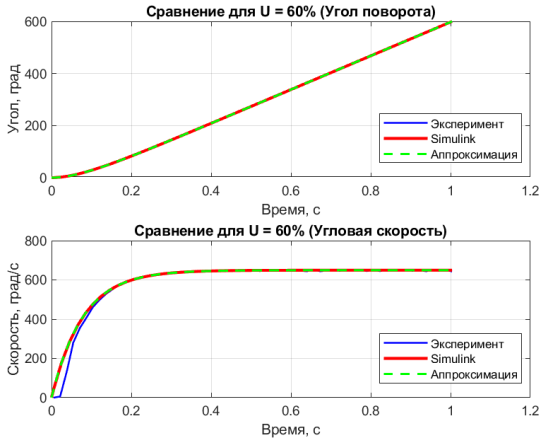
Рис. 13: Сравнение результатов для отрицательных напряжений. Хорошее совпадение кривых подтверждает адекватность выбранной математической модели.



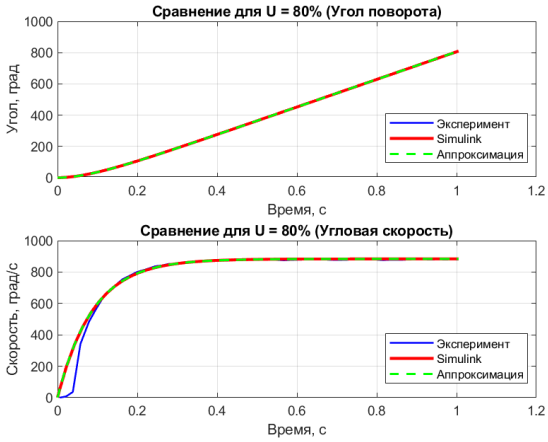
(a) Сравнение при +20%



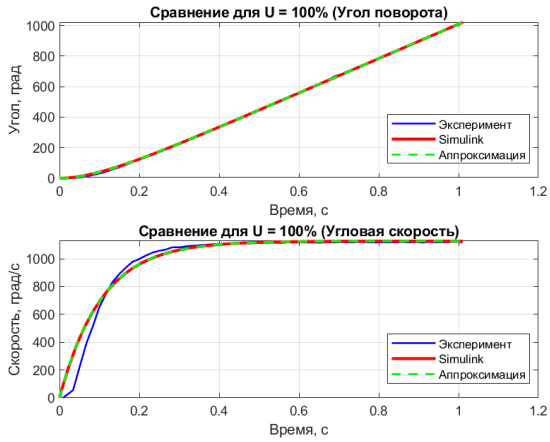
(b) Сравнение при +40%



(c) Сравнение при +60%



(d) Сравнение при +80%



(e) Сравнение при +100%

Рис. 14: Сравнение результатов для положительных напряжений. Хорошее совпадение кривых подтверждает адекватность выбранной математической модели.

6 Выводы

В ходе выполнения лабораторной работы все поставленные задачи были успешно решены:

1. Проведены эксперименты по снятию переходных процессов двигателя EV3 для 10 уровней управляющего сигнала в диапазоне от -100% до $+100\%$.

2. Определены параметры математической модели апериодического звена первого порядка. Для положительных напряжений средний статический коэффициент передачи составил $k \approx 0.18$ рад/(с·%), а средняя постоянная времени $T_m \approx 0.081$ с.

3. Выявлена нелинейность исследуемого объекта: параметры модели k и T_m не являются постоянными и зависят от величины управляющего сигнала. Это указывает на то, что модель первого порядка является лишь приближением, достаточным для многих инженерных задач, но не описывающим поведение системы во всем диапазоне с абсолютной точностью.

4. Построенная в Simulink модель с использованием средних параметров показала хорошее совпадение с экспериментальными данными, что подтверждает адекватность выбранного подхода для моделирования динамики двигателя EV3.

Таким образом, цель работы достигнута: на основе экспериментальных данных построена и верифицирована математическая модель двигателя EV3.

7 Программы

7.1 Программа EV3

```python
from ev3dev2.motor import LargeMotor, OUTPUT_B
import time

motorA = LargeMotor(OUTPUT_B)
voltages = [100, 80, 60, 40, 20, -20, -40, -60, -80, -100]

try:
    for vol in voltages:
        timeStart = time.time()
        startPos = motorA.position
        name = "data" + str(vol) + ".txt"
        file = open(name, "w")

        while True:
            timeNow = time.time() - timeStart
            motorA.run_direct(duty_cycle_sp=vol)
            pos = motorA.position - startPos
            file.write(str(timeNow) + " " + str(pos) + " " + str(motorA.speed) + "\n")

            if timeNow > 1:
                motorA.run_direct(duty_cycle_sp=0)
                break

            time.sleep(0.01)


        file.close()
        motorA.stop(stop_action='brake')
        time.sleep(0.4)

except Exception as e:
    raise e

finally:
    motorA.stop(stop_action='brake')


```

7.2 MATLAB Program

```MATLAB
clear all; close all; clc;
data_folder = '../data';
graphs_folder = 'graphs';
folders = {graphs_folder, fullfile(graphs_folder, 'angles'), fullfile(graphs_folder, 'velocities'), ...
    fullfile(graphs_folder, 'comparison'), fullfile(graphs_folder, 'parameters')};
[angles_folder, velocities_folder, comparison_folder, parameters_folder] = deal(folders{2:5});

voltages = [-100, -80, -60, -40, -20, 20, 40, 60, 80, 100];
n_files = length(voltages);
J = 0.0023; R = 6.5; U_max = 9.0;
[k_all, Tm_all, ke_all, km_all] = deal(NaN(n_files, 1));
```

```
%% Main processing loop
fig_angle = figure(1); fig_vel = figure(2);
for i = 1:n_files
    U_pr = voltages(i);
    filename = fullfile(data_folder, sprintf('data%d', U_pr));

    try
        data = readmatrix(filename);
        time = data(:, 1);
        angle_deg = data(:, 2);
        omega_deg = data(:, 3);
        angle_rad = angle_deg * pi / 180;
        omega_rad = omega_deg * pi / 180;

        % Subplots for angles and velocities
        figure(fig_angle); subplot(2, 5, i); plot(time, angle_deg, 'b.', 'MarkerSize', 6);
        xlabel('Time, s'); ylabel('Angle, rad'); title(sprintf('U = %d%%', U_pr)); grid on;

        figure(fig_vel); subplot(2, 5, i); plot(time, omega_deg, 'b.', 'MarkerSize', 6);
        xlabel('Time, s'); ylabel('Velocity, rad/s'); title(sprintf('U = %d%%', U_pr)); grid on;

        % Fitting by angle
        fun_theta = @(par, t) U_pr * par(1) * (t - par(2) * (1 - exp(-t/par(2))));
        options = optimoptions('lsqcurvefit', 'Display', 'off', 'MaxFunctionEvaluations', 10000);

        try
            par = lsqcurvefit(fun_theta, [15, 0.06], time, angle_rad, [], [], options);
            k = par(1); Tm = par(2);
            k_all(i) = k; Tm_all(i) = Tm;
            ke_all(i) = U_max / (100 * k);
            km_all(i) = J * R / (Tm * ke_all(i));

            time_apr = linspace(0, max(time), 200);
            theta_apr_deg = fun_theta([k, Tm], time_apr) * 180 / pi;
            omega_apr_deg = k * U_pr * (1 - exp(-time_apr/Tm)) * 180 / pi;

            figure(fig_angle); subplot(2, 5, i); hold on; plot(time_apr, theta_apr_deg, 'r-', 'LineWidth', 2);
            legend('Experiment', 'Approximation', 'Location', 'best');

            figure(fig_vel); subplot(2, 5, i); hold on; plot(time_apr, omega_apr_deg, 'r-', 'LineWidth', 2);
            legend('Experiment', 'Approximation', 'Location', 'best');

            fprintf('U = %d%%: k = %.4f, Tm = %.4f s, ke = %.4f, km = %.4f\n', U_pr, k, Tm, ke_all(i), km_all(i));
        catch ME
            fprintf('Approximation error U = %d%%: %s\n', U_pr, ME.message);
        end
    catch ME
        fprintf('Error reading %s: %s\n', filename, ME.message);
    end
```

```
end

saveas(fig_angle, fullfile(graphs_folder, 'all_angles.png'));
saveas(fig_vel, fullfile(graphs_folder, 'all_velocities.png'));

%% Individual plots for each voltage
for i = 1:n_files
    U_pr = voltages(i);
    if ~isnan(k_all(i)) && ~isnan(Tm_all(i))
        filename = fullfile(data_folder, sprintf('data%d', U_pr));
        try
            data = readmatrix(filename);
            time = data(:, 1);
            time_apr = 0:0.01:max(time);

            % Angle plot
            figure(10 + i);
            plot(time, data(:, 2), 'b.', 'MarkerSize', 8);
            hold on;
            theta_apr_deg = U_pr * k_all(i) * (time_apr - Tm_all(i) * (1 - exp(-time_apr/Tm_all(i)))) * 180 / pi;
            plot(time_apr, theta_apr_deg, 'r-', 'LineWidth', 2);
            xlabel('Time, s'); ylabel('Rotation angle, deg');
            title(sprintf('U = %d%%, k = %.3f, Tm = %.3f s', U_pr, k_all(i), Tm_all(i)));
            legend('Experiment', 'Approximation'); grid on;
            saveas(gcf, fullfile(angles_folder, sprintf('angle_%d.png', U_pr)));

            % Velocity plot
            figure(20 + i);
            plot(time, data(:, 3), 'b.', 'MarkerSize', 8);
            hold on;
            omega_apr_deg = k_all(i) * U_pr * (1 - exp(-time_apr/Tm_all(i))) * 180 / pi;
            plot(time_apr, omega_apr_deg, 'r-', 'LineWidth', 2);
            xlabel('Time, s'); ylabel('Angular velocity, deg/s');
            title(sprintf('U = %d%%, k = %.3f, Tm = %.3f s', U_pr, k_all(i), Tm_all(i)));
            legend('Experiment', 'Approximation'); grid on;
            saveas(gcf, fullfile(velocities_folder, sprintf('velocity_%d.png', U_pr)));
        catch
            fprintf('Error plotting for U = %d%%\n', U_pr);
        end
    end
end

%% Parameter plots
valid_idx = ~isnan(k_all) & ~isnan(Tm_all);
omega_ust_rad = k_all .* voltages' * 180 / pi;

plots_data = {{voltages(valid_idx), omega_ust_rad(valid_idx), 'bo', 'Steady-state velocity \omega_{ss}, deg/s', 'omega_steady.png', true}, ...
              {voltages(valid_idx), Tm_all(valid_idx) * 1000, 'rs', 'Time constant T_m, ms', 'Tm.png', false}, ...
              {voltages(valid_idx), k_all(valid_idx) * 180/pi, 'gs', 'k, deg/(·s%%)', 'k.png', false}};

for p = 1:3
```

```
    figure(100 + p);
    plot(plots_data{p}{1}, plots_data{p}{2}, plots_data{p}{3}, 'MarkerSize', 10, 'MarkerFaceColor', plots_data{p}{3}(1));
    xlabel('Voltage U, %'); ylabel(plots_data{p}{4});
    grid on;
    if plots_data{p}{6}
        hold on; pp = polyfit(plots_data{p}{1}, plots_data{p}{2}, 1);
        plot([-100, 100], polyval(pp, [-100, 100]), 'r-', 'LineWidth', 2);
        legend('Experiment', sprintf('y = %.2fx + %.2f', pp(1), pp(2)));
    end
    saveas(gcf, fullfile(parameters_folder, plots_data{p}{5}));
end

%% Simulink simulation
if license('test', 'Simulink')
    modelName = 'temp_motor_model';
    if bdIsLoaded(modelName)
        close_system(modelName, 0);
    end
    if isfile([modelName '.slx'])
        delete([modelName '.slx']);
    end
    create_motor_sim_model(modelName);
    sim_results = struct();
    for i = 1:n_files
        U_pr = voltages(i);
        if ~isnan(k_all(i)) && ~isnan(Tm_all(i))
            k_sim = k_all(i);
            Tm_sim = Tm_all(i);
            U_sim = U_pr;

            % Get simulation time from data
            filename = fullfile(data_folder, sprintf('data%d', U_pr));
            data = readmatrix(filename);
            time_exp = data(:, 1);
            t_end = max(time_exp);

            set_param(modelName, 'StopTime', num2str(t_end));

            try
                % Run simulation
                simOut = sim(modelName);
                pause(0.3);
                if ~isfile('omega_data.mat') || ~isfile('theta_data.mat')
                    fprintf('MAT files not found\n');
                    continue;
                end
                omega_mat = load('omega_data.mat');
                theta_mat = load('theta_data.mat');
                omega_fields = fieldnames(omega_mat);
                omega_var = omega_mat.(omega_fields{1});
                theta_fields = fieldnames(theta_mat);
                theta_var = theta_mat.(theta_fields{1});
                time_sim = omega_var.Time(:);
                omega_sim = omega_var.Data(:);
                theta_sim = theta_var.Data(:);

                key = sprintf('U_%d', U_pr);
                sim_results.(key).time = time_sim;
                sim_results.(key).omega = omega_sim;
```

```
                sim_results.(key).theta = theta_sim;

                %%%%% Individual comparison figure
                figure(200 + i);

                subplot(2,1,1);
                plot(time_exp, data(:,2), 'b.', 'MarkerSize', 6, 'DisplayName', 'Experiment');
                hold on;
                plot(time_sim, theta_sim * 180/pi, 'r-', 'LineWidth', 2, 'DisplayName', 'Simulink');

                time_apr = linspace(0, t_end, 200);
                theta_apr_rad = U_pr * k_sim * (time_apr - Tm_sim * (1 - exp(-time_apr/Tm_sim)));
                theta_apr_deg = theta_apr_rad * 180 / pi;
                plot(time_apr, theta_apr_deg, 'g--', 'LineWidth', 1.5, 'DisplayName', 'Approximation');

                xlabel('Time, s');
                ylabel('Angle, deg');
                title(sprintf('Comparison for U = %d%% (Rotation angle)', U_pr));
                legend('Location', 'best');
                grid on;

                subplot(2,1,2);
                plot(time_exp, data(:,3), 'b.', 'MarkerSize', 6, 'DisplayName', 'Experiment');
                hold on;
                plot(time_sim, omega_sim * 180/pi, 'r-', 'LineWidth', 2, 'DisplayName', 'Simulink');
                plot(time_apr, k_sim * U_pr * (1 - exp(-time_apr/Tm_sim)) * 180/pi, 'g--', 'LineWidth', 1.5, 'DisplayName', 'Approximation');

                xlabel('Time, s');
                ylabel('Angular velocity, deg/s');
                title(sprintf('Comparison for U = %d%% (Angular velocity)', U_pr));
                legend('Location', 'best');
                grid on;

                saveas(gcf, fullfile(comparison_folder, sprintf('comparison_%d.png', U_pr)));
                close(gcf);

                if isfile('omega_data.mat')
                    delete('omega_data.mat');
                end
                if isfile('theta_data.mat')
                    delete('theta_data.mat');
                end

            catch ME
                fprintf('Simulation error: %s\n', ME.message);
            end
        end
    end

    if bdIsLoaded(modelName)
```

```
        close_system(modelName, 0);
    end

    %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
    %% TODO comparison between experiment and simulation
    %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

    figure(310);
    colors = lines(n_files);
    for j = 1:n_files
        U_pr = voltages(j);
        if ~isnan(k_all(j)) && ~isnan(Tm_all(j))
            filename = fullfile(data_folder, sprintf('data%d', U_pr));
            data = readmatrix(filename); time_exp = data(:, 1);
            key = sprintf('U_%d', U_pr);
            if isfield(sim_results, key)
                time_sim = sim_results.(key).time;
                plot(time_exp, data(:,2), '.', 'Color', colors(j,:), 'MarkerSize', 4);
                hold on;
                plot(time_sim, sim_results.(key).theta * 180/pi, '-', 'Color', colors(j,:), 'LineWidth', 1.5, ...
                    'DisplayName', sprintf('U = %d%%', U_pr));
            end
        end
    end
    xlabel('Time, s'); ylabel('Angle, deg');
    title('Comparison of experiment and Simulink (angle)'); legend('Location', 'best'); grid on;
    saveas(gcf, fullfile(comparison_folder, 'all_angles_comparison.png')); close(gcf);

    figure(311);
    for j = 1:n_files
        U_pr = voltages(j);
        if ~isnan(k_all(j)) && ~isnan(Tm_all(j))
            filename = fullfile(data_folder, sprintf('data%d', U_pr));
            data = readmatrix(filename); time_exp = data(:, 1);
            key = sprintf('U_%d', U_pr);
            if isfield(sim_results, key)
                plot(time_exp, data(:,3), '.', 'Color', colors(j,:), 'MarkerSize', 4);
                hold on;
                plot(sim_results.(key).time, sim_results.(key).omega * 180/pi, '-', 'Color', colors(j,:), 'LineWidth', 1.5, ...
                    'DisplayName', sprintf('U = %d%%', U_pr));
            end
        end
    end
    xlabel('Time, s'); ylabel('Velocity, deg/s');
    title('Comparison of experiment and Simulink (velocity)'); legend('Location', 'best'); grid on;
    saveas(gcf, fullfile(comparison_folder, 'all_velocities_comparison.png'); close(gcf);

else
    fprintf('Simulink not available, skipping simulation.\n');
end

results = table(voltages', k_all, Tm_all, ke_all, km_all, ...
```

```
    'VariableNames', {'Voltage', 'k_rad_per_s_per', 'Tm_s', 'ke_V_s_per_rad', 'km_N_m_per_A'});
writetable(results, fullfile(graphs_folder, 'all_results.csv'));

function create_motor_sim_model(modelName)
    new_system(modelName);
    open_system(modelName);
    set_param(modelName, 'StopTime', '1.0', 'Solver', 'ode45');

    blocks = {'U', 'Gain_k', 'Sum', 'Gain_1Tm', 'Integ_omega', 'Integ_theta', 'ToFile_omega', 'ToFile_theta'};
    block_types = {'simulink/Sources/Constant', 'simulink/Math Operations/Gain', 'simulink/Math Operations/Sum', ...
                   'simulink/Math Operations/Gain', 'simulink/Continuous/Integrator', 'simulink/Continuous/Integrator', ...
                   'simulink/Sinks/To File', 'simulink/Sinks/To File'};
    params = {{}, {'Gain', 'k_sim'}, {'Inputs', '+-'}, {'Gain', '1/Tm_sim'}, ...
              {'InitialCondition', '0'}, {'InitialCondition', '0'}, ...
              {'Filename', 'omega_data.mat', 'Decimation', '1'}, ...
              {'Filename', 'theta_data.mat', 'Decimation', '1'}};

    for i = 1:length(blocks)
        add_block(block_types{i}, [modelName '/' blocks{i}]);
        for j = 1:2:length(params{i})
            set_param([modelName '/' blocks{i}], params{i}{j}, params{i}{j+1});
        end
    end

    add_line(modelName, 'U/1', 'Gain_k/1'); add_line(modelName, 'Gain_k/1', 'Sum/1');
    add_line(modelName, 'Integ_omega/1', 'Sum/2'); add_line(modelName, 'Sum/1', 'Gain_1Tm/1');
    add_line(modelName, 'Gain_1Tm/1', 'Integ_omega/1'); add_line(modelName, 'Integ_omega/1', 'Integ_theta/1');
    add_line(modelName, 'Integ_omega/1', 'ToFile_omega/1'); add_line(modelName, 'Integ_theta/1', 'ToFile_theta/1');
    save_system(modelName);
end
```